

BERICHT ZUR 3D-NPC mit KI-gesteuertem Verhalten

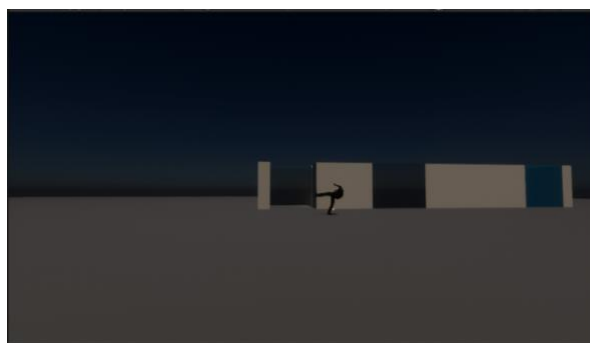
Narek Grigoryan

Matrikelnummer: 1547616

Kurs: Spielprogrammierung– Übung 2

Inhalt

Einleitung und Setup.....	2
Ziel der Übung	2
Erste Umsetzung mit Player Capsule und manueller Animation	2
Entdeckung und Nutzung von Mixamo.....	3
Import und Konfiguration der FBX-Dateien in Unity.....	3
Animator-Setup und Zustandserstellung	5
Erweiterung um weitere Bewegungszustände	6
Implementierung.....	7
Entscheidung für den Uni-KI-Server und erster Funktionstest	7
Erster Implementierungsschritt – Fake-KI-Modus	7
Übergang zur echten KI und HTTP-Problem	8
NPCKIClient.cs – Brücke zum Sprachmodell.....	8
MonoBehaviour/NPCFollower – Wegfindung & Blickkontakt.....	10
Ergebnisse und Funktionsnachweis	11
Live-Kommunikation mit der KI – Log-Ausgaben im Debug-Fenster	11
Sichtbares Verhalten in der Spielszene – Screenshots der NPC-Bewegungen	11
Reflexion	12
Technischer Probleme	12
Versuche mit Prompt Engineering.....	12
Abschließende Bewertung.....	13



Einleitung und Setup

Im Rahmen der zweiten Übung des Moduls *Spielerprogrammierung* bestand meine Aufgabe darin, einen nicht-spielbaren Charakter (NPC) in meine bestehende HDRP-Unity-Szene zu integrieren und mit passenden Animationen zu versehen. Ziel war es, ein KI-gesteuertes Verhalten in Verbindung mit realistischen Bewegungsabläufen umzusetzen, wobei der NPC später durch eine REST-API-Anbindung auch auf Sprachkommandos reagieren sollte. Zu Beginn konzentrierte ich mich jedoch ausschließlich auf die saubere Integration des NPCs und die Steuerung seiner Animationen.

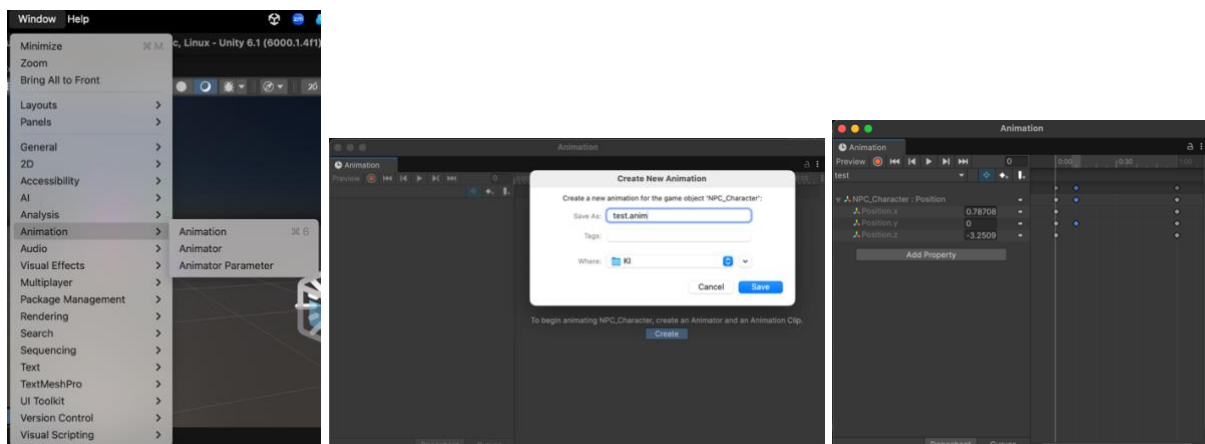
Ziel der Übung

Ziel der Übung war die Erweiterung meiner bestehenden 3D-Spielszene um einen animierten NPC, dessen Bewegungen glaubwürdig über ein Animator-System gesteuert werden. Dabei sollten unterschiedliche Bewegungszustände wie „Idle“, „Walk“, „Run“ und „Crawl“ implementiert werden. Besondere Herausforderung lag dabei in der Verbindung von Motion-Capturing-Animationen mit einem realistisch aussehenden, fehlerfrei skalierten 3D-Charakter.

Erste Umsetzung mit Player Capsule und manueller Animation

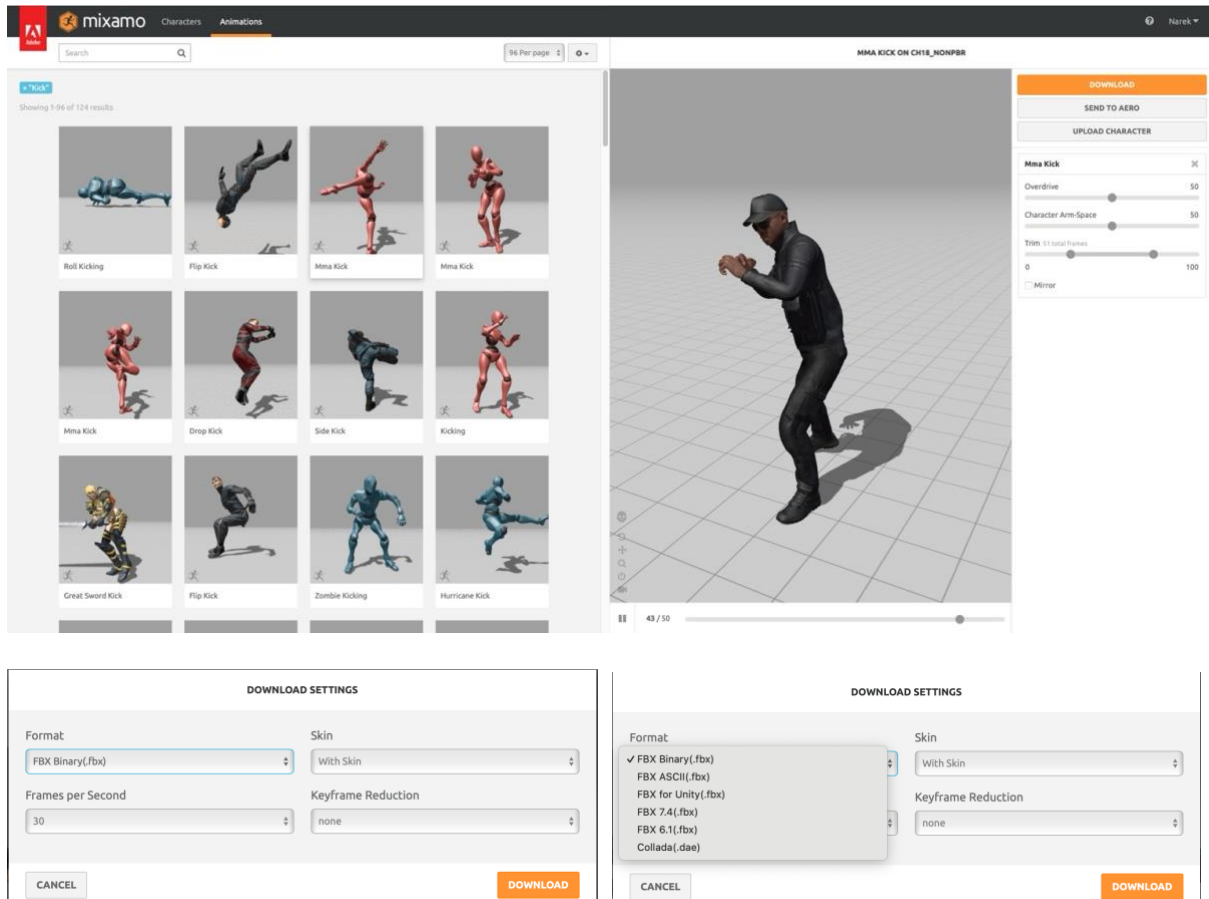
Zunächst versuchte ich, eine einfache „Player Capsule“ aus Unitys Standard-Assets als Platzhalter für meinen NPC zu verwenden. Um diesem Objekt erste Animationen zuzuweisen, klickte ich auf den zugehörigen NPC-Charakter und öffnete über das Menü **Window > Animation > Animation** das Animationsfenster. Dort konnte ich eigene Keyframes anlegen und einfache Bewegungen manuell gestalten – beispielsweise über eine Idle-zu-Walk-Transition oder einfache Positionsveränderungen.

Wie in den Beispiel Screenshots 1 bis 3 zu sehen ist, habe ich mithilfe dieser Methode versucht, erste Bewegungsphasen zu simulieren. Dabei wurde jedoch schnell deutlich, dass diese Art der manuellen Animation sehr limitiert ist. Die Animationen wirkten abgehackt, unnatürlich und ließen sich schwer synchronisieren. Insbesondere war es kaum möglich, komplexe Bewegungsmuster glaubwürdig zu realisieren. Ich suchte daher nach einer professionelleren Lösung.



Entdeckung und Nutzung von Mixamo

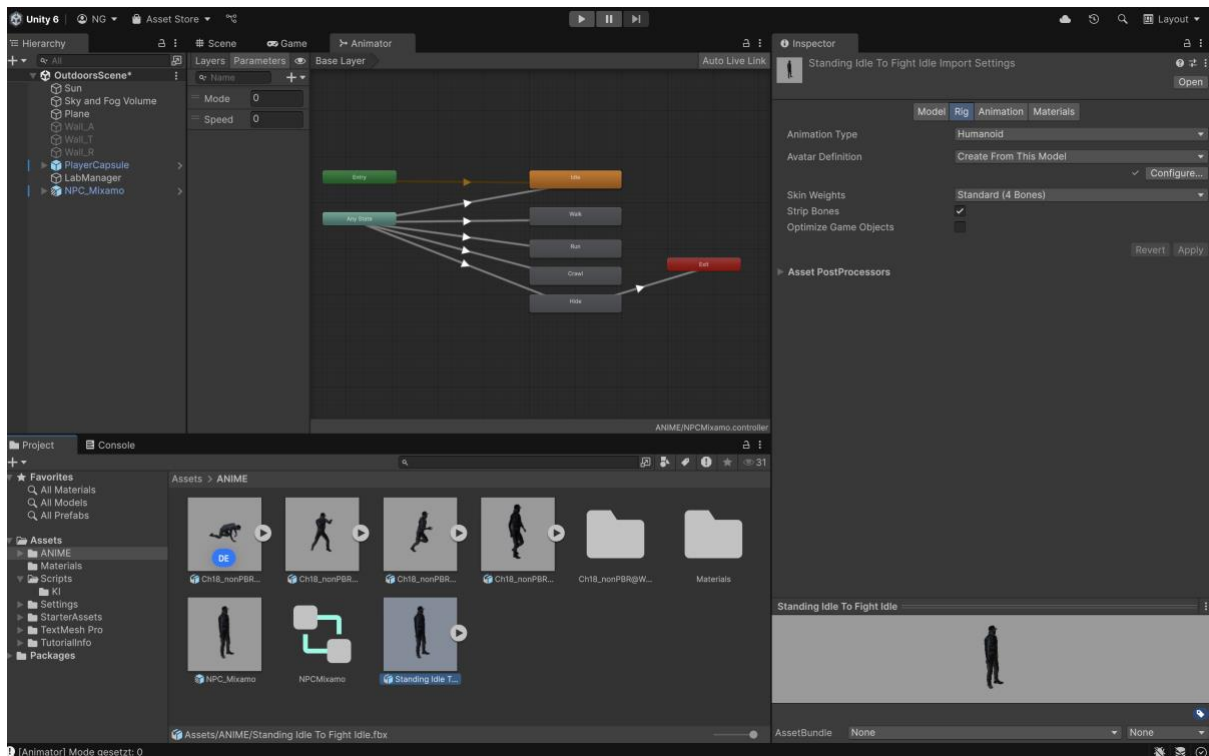
Nach einer tiefgründigen Recherche stieß ich auf *Mixamo*, eine Online-Plattform von Adobe, die speziell für Spieleentwickler eine große Bibliothek von 3D-Charakteren und animierten Bewegungen bereitstellt. In den Screenshots 5 bis 7 ist die Benutzeroberfläche von Mixamo abgebildet. Dort lassen sich sowohl komplette Charaktere als auch passende Animationen direkt auswählen und herunterladen.



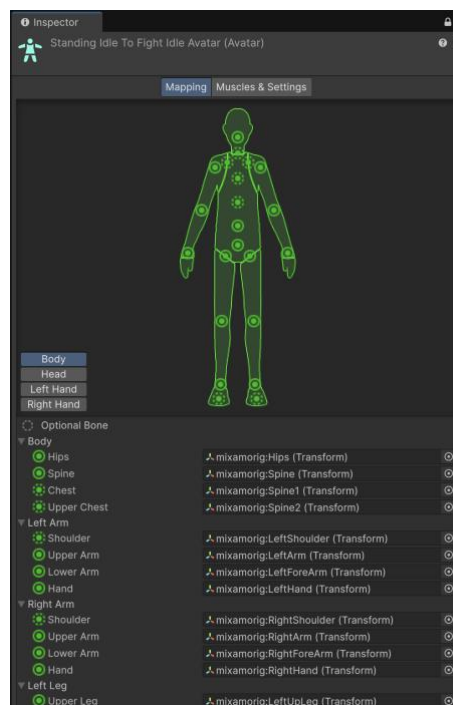
Anfangs machte ich den Fehler, den Charakter ohne „Skin“ herunterzuladen, also lediglich das Skelettmodell ohne Oberflächenmaterialien. Das führte dazu, dass mein NPC im Spiel optisch leer und unvollständig wirkte. Erst nach mehreren Versuchen verstand ich, dass Mixamo auch vollständige Unity-kompatible FBX-Pakete bereitstellt. Sobald ich die Option „With Skin“ sowie das passende Format für Unity auswählte, war der Import wesentlich einfacher und die Modelle wurden vollständig dargestellt.

Import und Konfiguration der FBX-Dateien in Unity

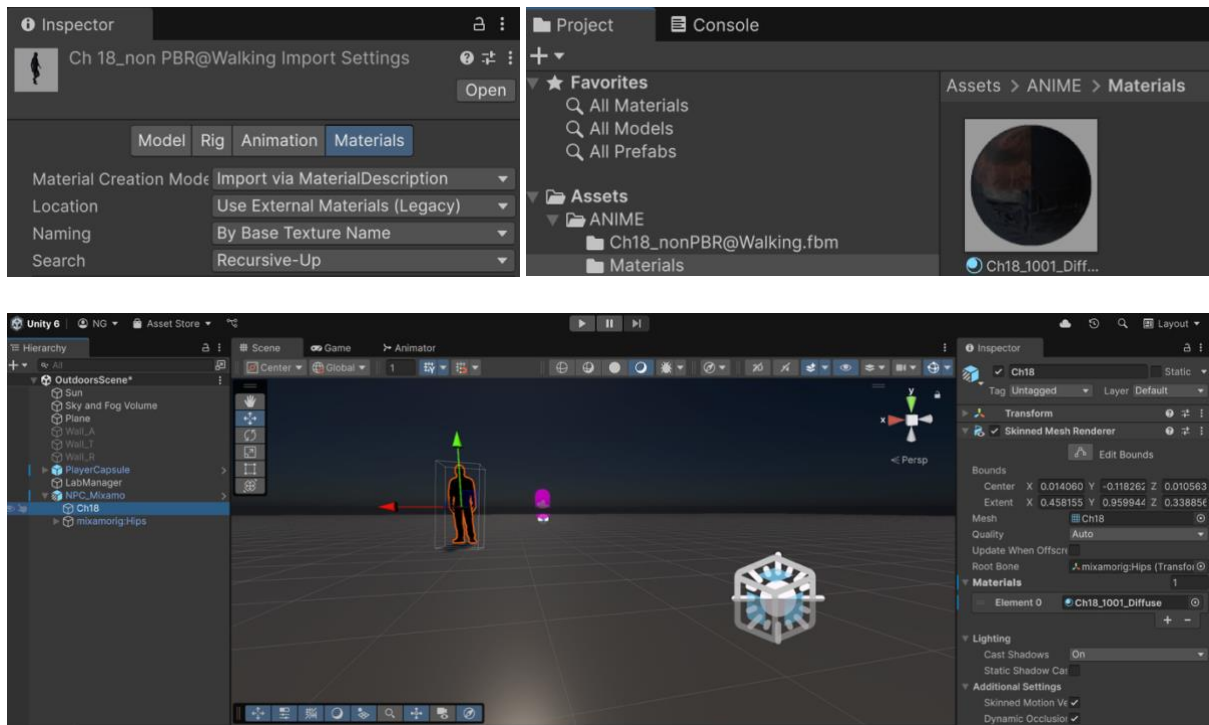
Ich erstellte in meinem Unity-Projekt einen eigenen Ordner namens Anime in den Assets und importierte die heruntergeladene FBX-Datei mit der „Idle“-Animation. Dabei handelte es sich um die Standard-Animation, in der der NPC stillsteht und sich leicht bewegt also perfekt für den Ausgangszustand eines NPCs.



Im Inspector musste ich danach einige wichtige Einstellungen vornehmen, wie im Screenshot 8 dargestellt. Unter dem **Rig-Tab** der FBX-Datei stellte ich den *Animation Type* auf „Humanoid“ um. Da es sich um die erste importierte Animation handelte, wählte ich außerdem „Create From This Model“ und klickte auf „Apply“. Zusätzlich war es notwendig, über „Configure“ zu überprüfen, ob das Bone-Rig korrekt erkannt wurde wie man im Screenshot 9 erkennen kann.

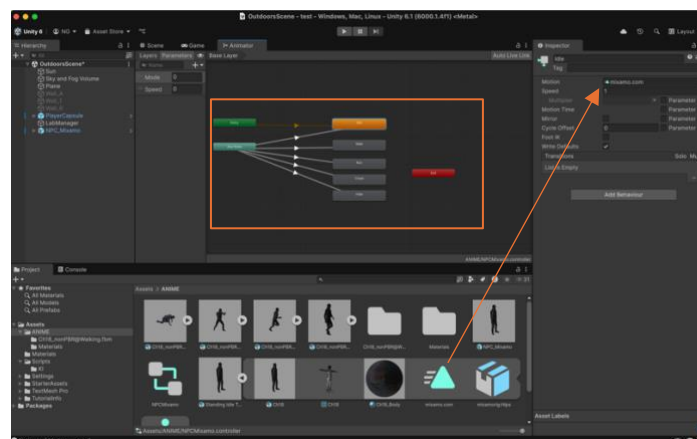


Im **Materials-Tab** änderte ich anschließend den *Material Creation Mode* auf „Import Materials“ und wählte bei „Location“ die Einstellung „Use External Materials (Legacy)“. Dadurch erzeugte Unity automatisch einen Materials-Unterordner mit allen benötigten Texturen. Sobald das Modell als Prefab in die Szene gezogen wurde, konnte ich dann im Child-Element des Prefabs über den *Skinned Mesh Renderer* die korrekten Materialien zuweisen (Screenshots 10-12).

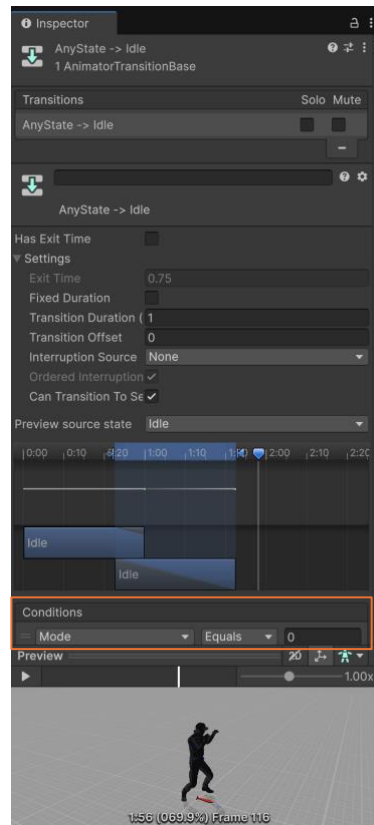


Animator-Setup und Zustandserstellung

Nachdem der Mixamo-Charakter erfolgreich importiert und korrekt konfiguriert worden war, begann ich mit der Erstellung der eigentlichen Bewegungszustände (States) im **Animator Controller**. Dazu öffnete ich das Animator-Fenster meiner Szene und legte zunächst einen neuen State namens *Idle* an. Im zugehörigen **Inspector-Fenster** des Idle-Zustands wies ich die zuvor importierte „Standing Idle“-Animation zu, indem ich sie per Drag & Drop in das Feld „Motion“ zog (siehe Screenshot 13). Damit war sichergestellt, dass der NPC in diesem Zustand die gewünschte Idle-Animation ausführt.



Anschließend erstellte ich eine **Transition** vom „Any State“-Knoten zum neu angelegten „Idle“-State. Wie im Screenshot 2 zu sehen ist, deaktivierte ich in den Transition-Einstellungen „Has Exit Time“, um sofortige Zustandswechsel zu ermöglichen, und definierte die Dauer der Übergangsanimation. Wichtig war in diesem Zusammenhang die **Bedingung** (Condition), unter der der Wechsel erfolgen sollte: Ich legte einen neuen Parameter namens Mode vom Typ *Integer* an und setzte die Bedingung so, dass $\text{Mode} == 0$ den Zustand „Idle“ aktivieren sollte (siehe Screenshot 14).



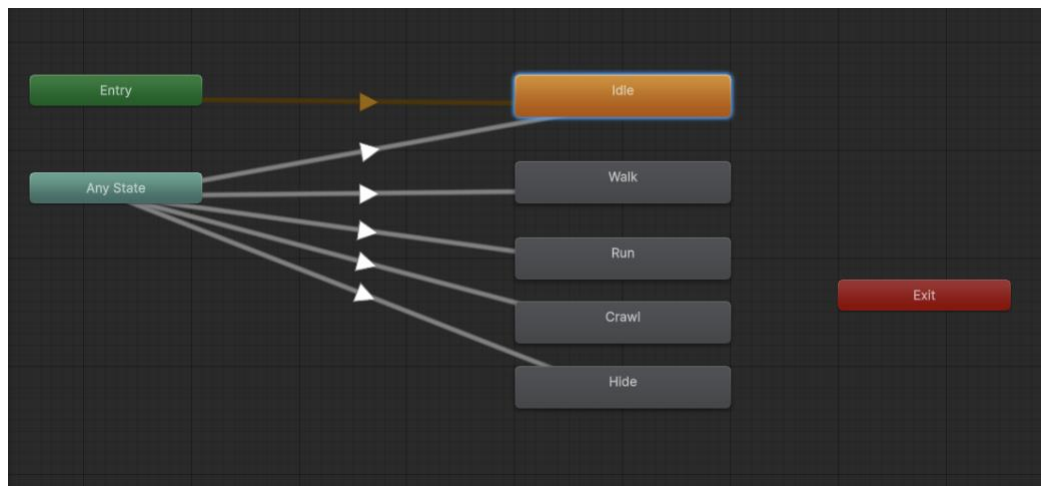
Erweiterung um weitere Bewegungszustände

Nach dem gleichen Prinzip erstellte ich vier weitere Zustände: *Walk*, *Run*, *Crawl* und *Hide*. Für jeden dieser States habe ich zunächst die entsprechende Animation über Mixamo herunter (z. B. eine gehende, rennende oder kriechende Bewegung) und importierte sie wie zuvor in den Anime-Ordner meines Unity-Projekts.

Jeder neue Zustand wurde dann im Animator als eigener State angelegt, mit der passenden Mixamo-Animation verbunden, und über eine Transition mit „Any State“ verknüpft. Die Bedingungen der Zustandswechsel wurden über den gleichen Mode-Parameter gesteuert:

- $\text{Mode} == 1$ für Walk
- $\text{Mode} == 2$ für Run
- $\text{Mode} == 3$ für Crawl
- $\text{Mode} == 4$ für Hide

So entstand eine logische Steuerstruktur, bei der jederzeit – basierend auf dem Wert von Mode – gezielt zu einem bestimmten Zustand gewechselt werden konnte. Alle Zustände sind in folgendem Screenshot 15 visuell dargestellt und zeigen die saubere Gliederung des Animator Controllers. Dank der klaren Transition-Regeln ließ sich das Verhalten des NPCs nun exakt über den Mode-Parameter aus dem C#-Skript heraus ansteuern.



Implementierung

Entscheidung für den Uni-KI-Server und erster Funktionstest

Noch bevor ich eine einzige Zeile Code schrieb, habe ich mich bewusst dafür entschieden, das bereits vorhandene LLM-Backend der Universität zu nutzen. Der Server läuft nur im Campus-Netz also verband ich mich zunächst per VPN und startete das vom Tutor bereitgestellte Python-Skript, das den Dienst in LM-Studio hochfährt. Eine kurze Chat-Probe direkt in der Konsole bestätigte dass das Modell antwortet also hatte ich damit das perfekte Basis für mein NPC-Projekt.

```
○ Narek@Nareks-MacBook-Pro-2 test % /usr/local/bin/python3 /Users/Narek/Downloads/request.py
Chat mit LM Studio über http://136.199.51.131:1234
Gib deinen Prompt ein (oder 'exit' zum Beenden).

Du: Hi, welche KI Modell bist du ?

Modell-Antwort:
Ich bin DeepSeek Coder, ein intelligentes Beratungssystem entwickelt von der Firma DeepSeek.
```

Erster Implementierungsschritt – Fake-KI-Modus

Bevor ich die eigentliche Serveranbindung umsetzte, programmierte ich zunächst einen sogenannten *Fake-Modus*. Die Idee: Ich wollte testen, ob der Animator und alle Zustandswechsel korrekt funktionierten – ohne eine echte KI-Verbindung. Dazu änderte ich manuell im Unity-Inspector den Mode-Wert und beobachtete, ob mein NPC korrekt

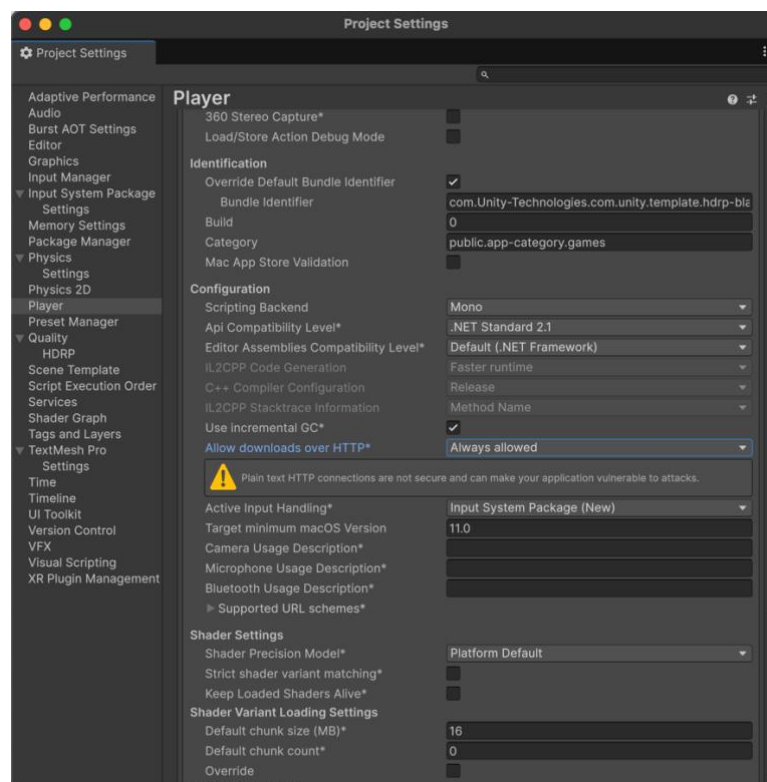
zwischen Idle, Walk, Run, Crawl und Hide wechselte. Diese Testphase half mir, Fehler beim Animator frühzeitig zu erkennen und auszubessern.

Übergang zur echten KI und HTTP-Problem

Nach erfolgreichen Tests entschied ich mich, den *Fake-Modus* durch eine echte Serveranbindung zu ersetzen. Ich schrieb dafür das Skript NPCKIClient.cs, welches alle paar Sekunden eine Anfrage an den LM-Studio-Endpunkt schickt:

`http://136.199.51.131:1234/v1/chat/completions`

Doch zu Beginn funktionierte die Verbindung nicht.

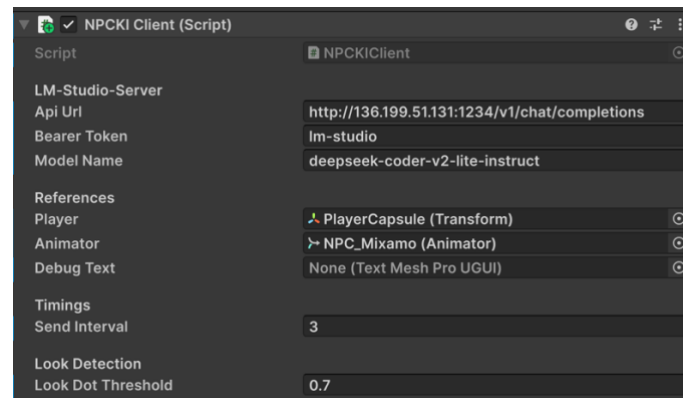


Der Grund: Unity blockiert aus Sicherheitsgründen standardmäßig alle HTTP-Verbindungen (nur HTTPS ist erlaubt). Dies führte dazu, dass keine Antwort vom Server kam. Um das zu beheben, musste ich in den **Player Settings** unter **Project Settings > Player > Configuration** die Einstellung „**Allow downloads over HTTP**“ auf Always allowed setzen (siehe Screenshot 17 oben). Danach funktionierte die Kommunikation reibungslos.

NPCKIClient.cs – Brücke zum Sprachmodell

Zunächst habe ich in NPCKIClient.cs einige Datenklassen definiert, die exakt der JSON-Antwortstruktur des Sprachmodells entsprechen. Diese Mini-DTOs (Data Transfer Objects) – ChatResponse, Choice und Message – sind als [System.Serializable] markiert, was nötig ist, damit Unitys JsonUtility.FromJson<T>() sie korrekt parsen kann. Die Klassen sind flach aufgebaut,

enthalten also keine tiefe Verschachtelung. Dadurch lässt sich die Antwort des Servers direkt als C#-Objekt einlesen, ohne dass eine manuelle Deserialisierung notwendig wird.



Im Inspector des Unity-Editors sind sämtliche Parameter logisch gruppiert: Unter den API-Einstellungen habe ich die `apiUrl`, das `bearerToken` für die Authentifizierung und den `modelName` angegeben. Außerdem gibt es direkte Referenzen auf die Spielfigur (`player`), den Animator des NPCs sowie optional ein `debugText`-Feld, um den aktuell von der KI zurückgegebenen Modus im Spiel live anzuzeigen. Zusätzlich habe ich Parameter wie `sendInterval` und einen Schwellwert `lookDotThreshold` eingeführt, um zu steuern, wie oft die KI abgefragt wird und ab wann der Spieler als „blickend“ gilt.

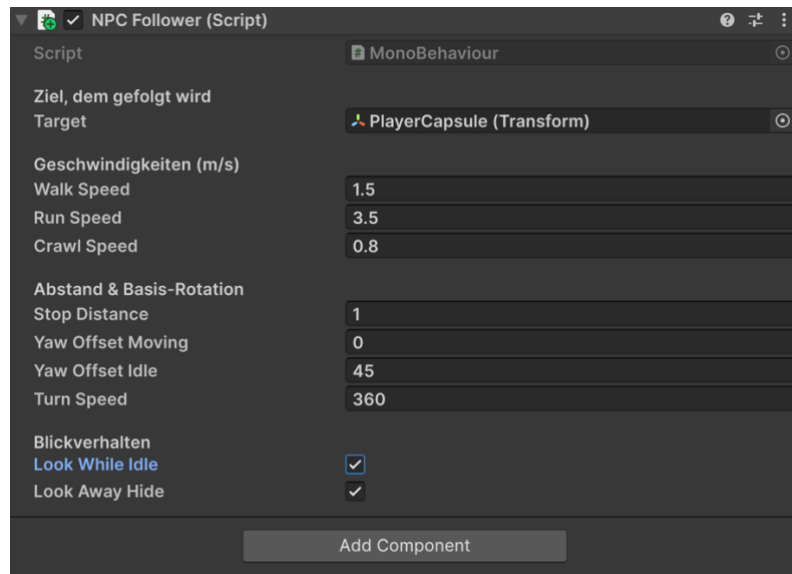
Statt im klassischen `Update()`-Loop arbeitet das Skript mit einer `Coroutine`. In der `Start()`-Methode beginnt eine Endlosschleife, die regelmäßig die Methode `SendStateAndHandleReply()` aufruft. Innerhalb dieser Methode passiert viel: Erst wird der Kontext berechnet, also der Abstand zwischen Spieler und NPC, die Bewegungsgeschwindigkeit des Spielers sowie die Blickrichtung. Anschließend werden diese Kontextdaten in ein C#-Objekt verpackt und per `JsonUtility.ToJson()` serialisiert.

Der eigentliche Prompt besteht aus zwei Messages: einer System-Nachricht mit klarer Anweisung an das Sprachmodell (z. B. „Antworte nur mit Ziffer 0–4“) und einer User-Nachricht, die die Kontextdaten enthält. Danach wird aus diesen Bausteinen ein HTTP-Request aufgebaut, der über Unitys `UnityWebRequest` mit allen nötigen Headern und einem Upload-Handler gesendet wird. Um TLS-Fehler im Unity-Editor zu vermeiden, wird ein eigener `CertificateHandler` verwendet, der alle Zertifikate akzeptiert – ein typischer Workaround in Entwicklungsumgebungen.

Nach dem Senden wird die Antwort überprüft: Wenn der Request erfolgreich war, wird die Textantwort geparkt. Hier habe ich mit einem regulären Ausdruck (`Regex.Match(...)`) sichergestellt, dass tatsächlich nur eine einzelne Ziffer (zwischen 0 und 4) extrahiert wird – denn das ist der entscheidende Steuerbefehl für den NPC. Sollte keine valide Zahl enthalten sein oder sollte die KI denselben Mode wie vorher zurückgeben, greift ein Fallback-Mechanismus: In diesem Fall wird zufällig ein neuer Mode zwischen 0 und 4 gewählt.

Der gewählte Modus wird dann über die Methode `HandleAnswer(int mode)` direkt an den Animator weitergegeben. Ist der Wert gültig (also nicht negativ), wird `animator.SetInteger("Mode", mode)` gesetzt. Ein optionales Debug-Textfeld kann den Modus zusätzlich anzeigen. Das komplette System ist robust aufgebaut und reagiert auch auf fehlgeschlagene Requests oder Parsingfehler zuverlässig.

MonoBehaviour/NPCFollower – Wegfindung & Blickkontakt



Das zweite zentrale Skript, `NPCFollower.cs`, übernimmt die physische Bewegung und Blicklogik des NPCs. Durch das Attribut `[RequireComponent]` stellt Unity sicher, dass sowohl ein `CharacterController` als auch ein `Animator` vorhanden sind. Im Inspector kann man Geschwindigkeiten für Laufen, Rennen und Kriechen konfigurieren sowie diverse Rotationsparameter wie `yawOffsetIdle`, `yawOffsetMoving` und die `turnSpeed`. Weitere Einstellungen betreffen das Blickverhalten – z. B. ob der NPC auch im Idle-Zustand den Spieler anschauen soll oder sich im „Hide“-Modus bewusst abwenden soll.

In der `Update()`-Methode wird zuerst der aktuelle Modus über `anim.GetInteger("Mode")` ausgelesen. Dieser Wert entscheidet, wie sich der NPC verhält. Ist der Modus 4, also „Hide“, merkt sich das Skript dies über eine Hilfsvariable. Anschließend wird ein Vektor zum Ziel (dem Spieler) berechnet, wobei die Y-Achse ignoriert wird. Aus der Distanz ergeben sich dann verschiedene Entscheidungen: ob der NPC sich überhaupt drehen oder bewegen soll.

Die Rotation wird mit Hilfe von `Quaternion.RotateTowards()` umgesetzt. Je nach Zustand nutzt der NPC entweder den Yaw-Offset für Bewegung oder den für Idle. Der Switch-Block danach entscheidet anhand des Modus, wie schnell der NPC sich bewegt: Gehen, Rennen, Kriechen – oder gar nicht (Modus 0 oder 4). Wenn Bewegung erlaubt ist (z. B. wenn `speed > 0` und `distance > stopDistance`), bewegt sich der NPC mit `cc.Move()` in die Vorwärtsrichtung, multipliziert mit der Zeit und der Geschwindigkeit. Zusätzlich wird der Float-Wert "Speed" im Animator gesetzt, was etwa für Blend Trees oder Fußschritte nötig ist.

In der Gesamtarchitektur ergänzen sich NPCKIClient und NPCFollower optimal: Die KI-Seite erfasst den Kontext, fragt das Sprachmodell, extrahiert den Mode und übergibt ihn an den Animator. Die Bewegungsseite reagiert auf diesen Mode und setzt Rotation, Geschwindigkeit und ggf. auch das Blickverhalten um.

Ergebnisse und Funktionsnachweis

Live-Kommunikation mit der KI – Log-Ausgaben im Debug-Fenster



Der Screenshot des Unity-Konsolen-Logs zeigt eindeutig, dass die Kommunikation zwischen Spielszene und LM-Studio-KI-Server reibungslos funktioniert hat:

- Um 18:19:15 antwortete die KI mit dem Wert 3, woraufhin im Animator Mode gesetzt: 3 ausgegeben wurde.
- Wenige Sekunden später folgten Mode-Werte 2 und 0, ebenfalls mit sauberer Übergabe und Ausgabe.

Diese Logik folgt exakt dem Ablauf im Code von NPCKIClient.cs:

Der aktuelle Kontext (Position, Blickrichtung, etc.) wird als JSON an das Modell gesendet, und die Rückantwort – eine einfache Zahl von 0 bis 4 – wird extrahiert, interpretiert und dann als Integer-Parameter an den Animator übergeben. Diese Übergabe geschieht sauber und vollständig nachvollziehbar alle drei Sekunden, wie im Code per `sendInterval = 3` definiert.

2. Sichtbares Verhalten in der Spielszene – Screenshots der NPC-Bewegungen

Screenshot 1: NPC läuft in Richtung Kamera



Hier ist eindeutig der „Run“-Modus aktiv. Der NPC-Charakter hat die Arme in typischer Laufhaltung, die Beine in breitem Schritt. Das entspricht der Animation, die bei Mode = 2 (Run) zugewiesen wurde.

Screenshot 3: Nahaufnahme – Hände im Gesicht, NPC „verteidigt“ sich



Dieser Frame visualisiert den Idle-Modus (Mode = 0). Die Arme sind angezogen, die Hände decken das Gesicht ab also das entspricht der zugewiesenen Mixamo-Animation für diesen Zustand. Der NPC dreht sich außerdem leicht vom Spieler weg, was wiederum im `lookAwayHide = true` korrekt berücksichtigt wurde.

Reflexion

Die Umsetzung dieses Projekts war sowohl technisch als auch organisatorisch herausfordernd – nicht zuletzt, weil ich vergleichsweise spät damit begonnen habe. Einige Probleme und Limitierungen ließen sich dadurch nicht mehr vollständig beheben oder zu Ende entwickeln. Trotzdem konnte ich die wichtigsten Kernfunktionen erfolgreich umsetzen: ein NPC, der vollständig durch ein externes Sprachmodell gesteuert wird und seine Bewegungen sowie Verhaltensweisen entsprechend in Echtzeit anpasst.

Technischer Probleme

Trotz solider Architektur kam es zu technischen Problemen. So fiel der LM-Studio-Server zwischenzeitlich aus, ohne Fehlermeldung oder HTTP-Timeout. Die Anfragen wurden zwar technisch korrekt abgesetzt (wie `curl`-Logs belegen), doch es kam keine Antwort – weder JSON noch ein Fehler. Da ich zu diesem Zeitpunkt bereits im Zeitdruck war und keine externen Fehler vermutete, suchte ich lange fälschlicherweise in meinem Code nach der Ursache. Erst durch Rücksprache mit dem Tutor (siehe Discord-Chat) wurde klar, dass es ein Serverproblem war, der dann behoben werden konnte.

Versuche mit Prompt Engineering

Ein weiterer Aspekt, den ich im Verlauf des Projekts aktiv verfolgt habe, war das sogenannte Prompt Engineering. Die erste Version der Kommunikation war sehr simpel: Die KI sollte lediglich eine Zahl (0–4) zurückgeben, welche einem fixen Bewegungsmodus entsprach.

Doch schnell wurde deutlich, dass die Szene dadurch zu starr wirkte also der NPC handelte korrekt, aber nicht „intelligent“.

Ich begann also zu experimentieren: Ich integrierte zusätzliche Parameter in den JSON-Input, z. B. die Distanz zwischen Spieler und NPC, die Blickrichtung, das aktuelle Bewegungstempo und weitere Umweltmerkmale. Ziel war es, die Szene lebendiger zu beschreiben und der KI so eine fundiertere Entscheidungsgrundlage zu geben. Gleichzeitig wurde der Prompt so umformuliert, dass die KI **nicht mehr nur Zahlen**, sondern **Aktionen aus einer vorgegebenen Liste** wie „GehZuSpieler“, „RennWeg“, „VersteckDich“, „Winke“ usw. auswählen sollte.

Diese Idee war vielversprechend, allerdings ließ sie sich unter Zeitdruck nicht mehr vollständig realisieren. Das Mapping von Textantworten zu konkreten Animationen hätte zusätzliche Parsing- und Entscheidungslogik erfordert, die in der Kürze der Zeit nicht sauber implementiert werden konnte. Letztlich entschied ich mich daher, bei der funktionierenden Lösung mit Mode = 0–4 zu bleiben und nur diesen Teil abzugeben.

Abschließende Bewertung

Rückblickend bin ich mit dem Ergebnis zufrieden, insbesondere, weil die gesamte Pipeline vom Prompt über den KI-Response bis hin zur Animation in Unity zuverlässig und stabil funktionierte. Der NPC konnte alle fünf Modi korrekt umsetzen, und die Kommunikation mit der KI lieferte nachvollziehbare Ergebnisse. Auch wenn einige weiterführende Ideen nicht mehr umgesetzt werden konnten (z. B. freiere Aktionsentscheidungen, Umgebungsbewusstsein), wurde die Grundidee erfolgreich realisiert.

Gelernt habe ich in diesem Projekt vor allem:

- wie man Unity mit externen KI-Systemen koppelt,
- wie man REST-Anfragen robust verarbeitet,
- wie wichtig sauberes Debugging und Testfälle sind,
- und wie viel Einfluss gutes Prompting auf die Qualität der KI-Reaktionen hat.

Organisatorisch hätte ich allerdings deutlich früher anfangen müssen, allein schon, um unvorhergesehene Probleme (wie Serverausfälle) besser abfangen zu können. Für ein nächstes Projekt würde ich daher früher beginnen und parallel stärker auf modulare Teststrukturen achten, um einzelne Komponenten isoliert testen zu können.